Focus — strutture dati: come funzionano, e come usarle meglio

> **Materiale facoltativo.** Non si dà per svolto e non serve per nessun esercizio del corso. È per chi programma già, o per chi, finita la lezione, si chiede *perché* le strutture si comportano così e se si possa scrivere lo stesso codice meglio: la risposta è sì.

Si dà per saputo tutto `L07`: i deck, gli esercizi e `L07_riepilogo.md`. Il focus ha tre parti, che si possono leggere separatamente:

- **I. Dentro le strutture**: che cosa fa davvero un `for`, quanto costa ogni operazione, che cosa vuol dire «copia»;
- **II. Strumenti in più**: insiemi, `Counter`, `defaultdict`, le comprehension, i generatori, e altro che rende il codice più corto senza renderlo più oscuro;
- **III. Dati veri**: la forma in cui arrivano i dati da un file o dal web.

Ogni esempio è eseguito con Python 3.14. Alcuni usano `import`, per prendere uno strumento da un modulo: i moduli arrivano in `L08`, e qui basta copiare la riga.

> **Una regola, prima di cominciare.** Ogni strumento di questa pagina ha un equivalente scritto con un ciclo, ed è quello che si impara per primo. Usa lo strumento quando il ciclo lo sapresti già scrivere: se non lo sapresti, lo strumento non ti sta facendo risparmiare tempo, ti sta nascondendo quello che succede.

Parte I — Dentro le strutture

Che cosa fa un `for`

Un `for` su una lista **avanza di una posizione a ogni giro**, e a ogni giro **rilegge la lunghezza** della lista. È equivalente a questo `while`:

```
indice = 0
while indice < len(numeri):
    numero = numeri[indice]
    ...                          # il corpo del for
    indice = indice + 1
```

Il `for` tiene un **riferimento alla lista** e un **indice**, non l'elemento. Questo spiega che cosa succede togliendo elementi mentre la si scorre:

```
numeri = [2, 4, 5, 6, 8]
for numero in numeri:
    if numero % 2 == 0:
        numeri.remove(numero)
print(numeri)                    # [4, 5, 8]
```

Tolto il `2`, il `4` scivola in posizione 0, ma l'indice è già passato a 1: il `4` non viene mai guardato. Lo stesso succede all'`8` dopo il `6`. E dopo due rimozioni la lista ha tre elementi, quindi il ciclo fa tre giri, non cinque.

L'iteratore, visto da vicino

Quello che il `for` usa per avanzare si chiama **iteratore**, e si può chiedere esplicitamente: `iter()` lo costruisce, `next()` gli chiede l'elemento successivo.

```
numeri = [2, 4, 5]
it = iter(numeri)
next(it)                        # 2
next(it)                        # 4
next(it)                        # 5
next(it)                        # StopIteration
```

Quando gli elementi finiscono, `next()` solleva `StopIteration`: è il segnale con cui il `for` capisce che deve fermarsi. Ogni `for` di Python è questo, dietro le quinte: un iteratore e una serie di `next()`.

Perché il dizionario si ferma

L'iteratore di un dizionario fa un controllo in più: si ricorda **quante chiavi c'erano all'inizio**, e a ogni passo controlla che siano ancora quelle. Se cambiano, si ferma:

```
minime = {'lun': 3, 'mar': -1, 'mer': 2}
for giorno in minime:
    if minime[giorno] < 0:
        del minime[giorno]
# RuntimeError: dictionary changed size during iteration
```

Sulla lista lo stesso errore è **silenzioso**, sul dizionario **rumoroso**. Il secondo comportamento è il migliore: un errore che si vede si corregge.

Costruire invece di togliere

Per tenere solo una parte degli elementi, si costruisce una struttura nuova:

```
dispari = []
for numero in numeri:
    if numero % 2 != 0:
        dispari.append(numero)
```

Togliere dalla lista originale si può, ma solo con un `while` che **avanza l'indice solo quando non toglie**, perché dopo una rimozione l'elemento successivo è già scivolato nella posizione corrente:

```
indice = 0
while indice < len(numeri):
    if numeri[indice] % 2 == 0:
        del numeri[indice]          # l'indice resta fermo
```

```
    else:
        indice = indice + 1
```

Funziona, ma conviene costruire, per tre ragioni:

- **è più semplice**: il codice dice che cosa tenere, non come spostare gli indici. La correttezza del `while` dipende tutta da quell'`else`, che è proprio il dettaglio che si sbaglia;
- **è molto più veloce**: ogni `del` fa scivolare indietro **tutti** gli elementi successivi. Su 100 000 numeri, togliendo i pari, il `while` impiega **0,64 secondi**, costruire una lista nuova **0,003**: circa 200 volte meno, e il divario cresce con la lunghezza;
- **l'originale resta**: se serve ancora, è lì. E se qualcun altro la usa con un secondo nome (`b = a`), togliere la modifica anche per lui.

Su un dizionario, se si vuole proprio modificare l'originale, si scorre **una copia delle chiavi**: `list(minime)` è una lista nuova, che non cambia mentre si toglie dal dizionario.

```
for giorno in list(minime):
    if minime[giorno] < 0:
        del minime[giorno]
```

Quanto costa ogni operazione

Le operazioni hanno costi diversi, e la differenza dipende da **quanti elementi l'operazione deve leggere**, non dal computer. Si dice che un'operazione è *costante* se ne legge sempre lo stesso numero, *lineare* se li legge tutti, o una parte che cresce con la lunghezza.

Operazione	Lista	Dizionario, insieme
raggiungere un elemento: `l[i]`, `d[k]`	costante	costante
`x in ...`	**lineare**: li legge uno per uno	costante
aggiungere in fondo: `append`, `d[k] = v`	costante	costante
inserire o togliere all'inizio: `insert(0, x)`, `pop(0)`	**lineare**: tutti gli altri scivolano	—
togliere per valore: `remove(x)`	**lineare**: prima cerca, poi fa scivolare	`del d[k]`: costante
ordinare: `sort()`, `sorted()`	un po' più che lineare	—

In pratica, con un milione di elementi: `999999 in lista` impiega circa **5 millisecondi**, `999999 in diz` circa **0,1 microsecondi**, decine di migliaia di volte meno. Diecimila ricerche: 48 secondi contro un millesimo di secondo.

Perché il dizionario va dritto alla chiave

Il dizionario **calcola dove deve stare** la chiave, invece di cercarla fra le altre. A ogni chiave Python associa un numero, il suo **hash**, e da quel numero ricava la posizione in cui il valore è conservato.

```
hash(42)                        # 42
hash((1, 2)) == hash((1, 2))    # True: stessa tupla, stesso hash
hash([1, 2])                    # TypeError: unhashable type: 'list'
```

Per questo **le chiavi devono essere immutabili**. Se una chiave cambiasse dopo essere stata inserita, il suo hash cambierebbe, il dizionario la cercherebbe nel posto sbagliato e non la ritroverebbe più. Una lista si può modificare, quindi Python non le calcola nemmeno l'hash.

L'hash delle stringhe cambia **a ogni esecuzione** di Python, per ragioni di sicurezza: `hash('ciao')` dà un numero diverso ogni volta che riapri l'interprete. Dentro una stessa esecuzione resta fisso, ed è quello che conta.

Identità e copie

`==` chiede se due strutture hanno **lo stesso contenuto**. `is` chiede se sono **la stessa struttura**, con due nomi.

```
a = [1, 2, 3]
b = a
c = list(a)
a == b, a is b                  # (True, True): due nomi, una lista
a == c, a is c                  # (True, False): stesso contenuto, due liste
```

`is` si usa quasi solo per `None`: `if risultato is None:`. Per confrontare valori si usa `==`.

La copia superficiale e quella profonda

`list(a)`, `a.copy()` e `a[:]` fanno tutte una copia **superficiale**: una lista nuova, con dentro **gli stessi elementi**. Se gli elementi sono numeri o stringhe, che non si modificano, non fa differenza. Se sono liste, la differenza c'è:

```
import copy

tabella = [[1, 2], [3, 4]]
superficiale = copy.copy(tabella)
profonda = copy.deepcopy(tabella)
tabella[0][0] = 9
superficiale                    # [[9, 2], [3, 4]]: le righe sono condivise
profonda                        # [[1, 2], [3, 4]]: le righe sono state copiate
```

`copy.deepcopy` copia **tutti i livelli**, per quanto sia annidata la struttura. Costa di più, e serve solo quando dentro ci sono strutture modificabili.

Una tupla che contiene una lista

Una tupla non si può modificare. Ma se contiene una lista, **la lista sì**:

```
t = ([1, 2], 3)
t[0].append(4)
t                               # ([1, 2, 4], 3)
t[0] = [5]                      # TypeError: 'tuple' object does not support item assignment
```

L'immutabilità riguarda **la tupla**, non gli oggetti che contiene: in ogni posizione resta sempre lo stesso oggetto, ma se quell'oggetto è una lista, la lista si può modificare. Per questo una tupla così non può fare da chiave di un dizionario:

```
{t: 'x'}                        # TypeError: cannot use 'tuple' as a dict key (unhashable type: '
    list')
```

Una tupla è immutabile davvero, e può fare da chiave, solo se è immutabile tutto quello che contiene.

Un solo meccanismo: i riferimenti

I tre casi di questa sezione hanno la stessa causa: **nomi, liste, tuple e dizionari contengono riferimenti agli oggetti, non gli oggetti**. Un riferimento dice dove si trova l'oggetto in memoria, e due riferimenti possono indicare lo stesso oggetto.

```
lista = [1, 2]
t = (lista, 3)
lista.append(4)                 # modifico la lista attraverso il suo nome
t                               # ([1, 2, 4], 3): la tupla contiene un riferimento a quella lista
t[0] is lista                   # True
```

Da qui vengono tutti e tre:

- **`b = a` non copia**, perché un nome è un riferimento, e `b` diventa un secondo riferimento allo stesso oggetto;
- **`list(tabella)` copia un livello solo**, perché la lista nuova contiene gli stessi riferimenti alle righe;
- **una tupla può contenere qualcosa che cambia**, perché è immutabile il riferimento, non l'oggetto a cui porta.

Quando una struttura sembra cambiare da sola, la domanda da farsi è: **chi altro ha un riferimento a questo oggetto?**

Parte II — Strumenti in più

`get` e `setdefault`

`diz.get(chiave, default)` restituisce il valore se la chiave c'è, e `default` se non c'è, **senza `KeyError` e senza modificare il dizionario**. Senza secondo argomento, il valore di ripiego è `None`.

Conviene quando una chiave mancante è un caso normale: il conteggio, `conteggio[x] = conteggio.get(x, 0) + 1`, ne è l'esempio tipico. Rispetto a `if x in conteggio:` seguito da `conteggio[x]`, il dizionario viene interrogato una volta invece di due.

Non conviene quando una chiave mancante è un errore, perché `get` la nasconde. Se il programma deve leggere la provincia `'MI'` e non c'è, è meglio un `KeyError` che si vede subito di un `None` che prosegue e salta fuori tre funzioni dopo, dove nessuno capisce da dove venga.

`diz.setdefault(chiave, default)` fa un passo in più: se la chiave non c'è, la **crea** con il valore `default`, e in ogni caso restituisce il valore. Raggruppare diventa una riga:

```
misure = [('gen', -2), ('gen', 1), ('feb', 3), ('gen', -4), ('feb', 0)]

perMese = {}
for mese, minima in misure:
    perMese.setdefault(mese, []).append(minima)
# {'gen': [-2, 1, -4], 'feb': [3, 0]}
```

È lo stesso risultato di `if mese not in perMese: perMese[mese] = []`. Il nome però non dice che cosa fa, e molti preferiscono la versione con l'`if`, o `defaultdict`, più sotto.

Le strutture vuote valgono `False`

In una condizione, una struttura **vuota** vale `False` e una non vuota `True`:

```
bool([]), bool({}), bool(''), bool(())        # (False, False, False, False)
bool([0]), bool(' ')                          # (True, True): un elemento c'è
```

Così `if lista:` vuol dire «se la lista ha almeno un elemento», e `if not diz:` «se il dizionario è vuoto». È più corto di `if len(lista) > 0:` e si legge allo stesso modo.

Con un'attenzione: valgono `False` anche `0`, `0.0` e `None`. Se una variabile può contenere `0` come valore legittimo, `if x:` lo tratta come se mancasse. In quel caso si scrive la condizione per intero: `if x is not None:`.

Gli insiemi

Un **insieme**, `set`, è una collezione **senza duplicati e senza ordine**. Si costruisce da qualunque sequenza:

```
set([3, 1, 3, 2, 1])          # {1, 2, 3}
```

Serve a tre cose:

- **togliere i duplicati**: in `parole = "piove piove la gatta non si muove la fiamma traballa la gatta fa la balla".split()` ci sono 15 parole, ma `len(set(parole))` dice che quelle diverse sono 10;
- **chiedere se un elemento c'è**: `x in insieme` è costante, come per le chiavi di un dizionario. Un insieme è, in effetti, un dizionario fatto di sole chiavi;
- **le operazioni degli insiemi**:

```
a = {1, 2, 3}
b = {2, 3, 4}
a | b                          # {1, 2, 3, 4}: unione
a & b                          # {2, 3}: intersezione
a - b                          # {1}: in a ma non in b
a ^ b                          # {1, 4}: in uno solo dei due
```

Tre trappole:

- **`{}` è un dizionario vuoto**, non un insieme. L'insieme vuoto si scrive `set()`;
- **l'ordine non è garantito.** Un insieme di numeri piccoli spesso esce ordinato, ma per caso; un insieme di stringhe esce in un ordine che **cambia da un'esecuzione all'altra**, perché dipende dall'hash. Se serve un ordine, `sorted(insieme)`;
- `insieme.remove(x)` dà `KeyError` se `x` non c'è; `insieme.discard(x)` no.

`Counter` e `defaultdict`

Il modulo `collections` ha due dizionari specializzati, che risparmiano proprio i due schemi che si scrivono più spesso.

`Counter` conta. Gli si passa una sequenza, e restituisce un dizionario che a ogni elemento associa quante volte compare:

```
from collections import Counter

conteggio = Counter(parole)
conteggio['gatta']             # 2
conteggio['cane']              # 0: una chiave che non c'è vale 0, senza KeyError
```

`defaultdict` crea la chiave al primo accesso. Gli si dice che cosa mettere quando una chiave manca: `list` per una lista vuota, `int` per lo zero.

```
from collections import defaultdict

perMese = defaultdict(list)
for mese, minima in misure:
    perMese[mese].append(minima)          # nessun if: la lista nasce da sola
```

Con `defaultdict(int)`, il conteggio diventa `conteggio[parola] += 1`.

La comodità ha un prezzo: **anche leggere una chiave mancante la crea**. Dopo `conta['cane']`, che vale `0`, `'cane' in conta` vale `True`. Se il dizionario serve anche a sapere quali chiavi ci sono, un `defaultdict` può

riempirsi di chiavi che nessuno ha mai inserito. Quando si è finito di costruirlo, `dict(perMese)` lo riporta a un dizionario normale.

Unire due dizionari

Da Python 3.9 due dizionari si uniscono con `|`:

```
d1 = {'lun': 3, 'mar': -1}
d2 = {'mar': 0, 'mer': 2}
d1 | d2                          # {'lun': 3, 'mar': 0, 'mer': 2}
d2 | d1                          # {'mar': -1, 'mer': 2, 'lun': 3}
```

Il risultato è un dizionario **nuovo**. Se una chiave è in tutti e due, **vince quello a destra**: per questo `d1 | d2` e `d2 | d1` danno a `'mar'` valori diversi. L'ordine delle chiavi è quello di inserimento: prima quelle del dizionario a sinistra.

Per modificare invece il dizionario che c'è, senza costruirne uno nuovo, `d1 |= d2`, o il metodo più vecchio `d1.update(d2)`: fanno la stessa cosa.

Le viste delle chiavi si comportano come insiemi, e rispondono alle domande sul confronto fra due dizionari:

```
d1.keys() & d2.keys()            # {'mar'}: le chiavi in comune
d1.keys() - d2.keys()            # {'lun'}: solo nel primo
```

Le comprehension

Una **comprehension** costruisce una struttura da un'altra, in una riga. Ce n'è una per le liste, una per i dizionari e una per gli insiemi. Gli esempi usano le minime di una settimana:

```
temperature = [3, -1, -4, 0, 2, 5, -2]
giorni = ['lun', 'mar', 'mer', 'gio', 'ven', 'sab', 'dom']
```

Struttura	Comprehension	Il ciclo equivalente
lista	`[t * 9 / 5 + 32 for t in temperature]`	`l = []`, poi `for t in temperature: l.append(t * 9 / 5 + 32)`
lista, con condizione	`[t for t in temperature if t < 0]`	lo stesso, con un `if` prima dell'`append`
dizionario	`{g: t for g, t in zip(giorni, temperature)}`	`d = {}`, poi `for g, t in zip(...): d[g] = t`
insieme	`{t % 2 for t in temperature}`	`s = set()`, poi `for t in temperature: s.add(t % 2)`

La seconda riga dà `[-1, -4, -2]`; la prima le stesse temperature in gradi Fahrenheit, `[37.4, 30.2, 24.8, 32.0, 35.6, 41.0, 28.4]`.

Si legge da sinistra a destra come una frase: **che cosa** mettere, **per ogni** elemento, **se** vale una condizione. Il ciclo equivalente c'è sempre, ed è quello da scrivere per primo: la comprehension è la forma corta di un ciclo che sai già scrivere.

Quando non conviene. Una comprehension con due `for` e una condizione entra in una riga, ma non si legge più:

```
[[riga[j] for riga in tabella] for j in range(len(tabella[0]))]
```

trasporre una tabella, scambiando righe e colonne. È corretta, ma chi la legge deve smontarla per capirla: due cicli scritti per esteso dicono la stessa cosa in un modo che si segue. La regola è semplice: **se la comprehension non si legge al primo colpo, scrivi il ciclo**.

La tupla che non c'è: i generatori

Con le parentesi tonde **non si ottiene una tupla**:

```
gen = (t for t in temperature if t < 0)
type(gen)                       # <class 'generator'>
```

È un'**espressione generatrice**: un oggetto che produce gli elementi **uno alla volta**, solo quando qualcuno li chiede, come fanno `range`, `enumerate` e `zip`. Per avere una tupla si passa a `tuple()`: `tuple(t for t in temperature if t < 0)` dà `(-1, -4, -2)`.

Un generatore occupa sempre lo stesso spazio, qualunque sia il numero di elementi che produce, perché non li conserva in memoria: li produce uno alla volta. Una lista di un milione di numeri occupa circa **8,4 MB**; il generatore equivalente, `(x for x in range(1000000))`, **200 byte**. Per questo, quando una sequenza deve essere scorsa una sola volta, il generatore basta:

```
sum(t for t in temperature if t < 0)        # -7, senza costruire la lista dei negativi
```

Un generatore si consuma. Una volta scorso, è vuoto:

```
gen = (t for t in temperature if t < 0)
sum(gen)                        # -7
sum(gen)                        # 0: gli elementi sono già stati prodotti
```

Una lista si può scorrere quante volte si vuole; un generatore una volta sola. Se servono due passate, ci vuole una lista.

`any` e `all`

`any(...)` vale `True` se **almeno un** elemento è vero, `all(...)` se lo sono **tutti**. Con un'espressione generatrice dentro, rispondono in una riga a domande che altrimenti chiedono un ciclo e una variabile:

```
any(t < 0 for t in temperature)             # True: c'è almeno un giorno sotto zero
all(t < 10 for t in temperature)            # True: tutti i giorni sotto i 10 gradi
```

`any` si ferma al primo elemento vero, `all` al primo falso: non guardano il resto. Su una struttura vuota, `any([])` vale `False` e `all([])` vale `True`: «tutti gli elementi di niente» è vero, perché nessuno lo smentisce.

`zip` che controlla, e lo spacchettamento esteso

`zip` si ferma alla sequenza più corta, senza avvisare: con due liste di lunghezza diversa, una parte dei dati sparisce. Da Python 3.10, con `strict=True`, se ne accorge:

```
list(zip(['a', 'b', 'c'], [1, 2], strict=True))
# ValueError: zip() argument 2 is shorter than argument 1
```

Quando le due liste **devono** avere la stessa lunghezza, conviene scriverlo sempre: è lo stesso principio del dizionario che si ferma con `RuntimeError`, un errore che si vede invece di un risultato sbagliato.

Lo **spacchettamento** di `L07` assegna un nome a ogni elemento, e vuole tanti nomi quanti elementi. Con l'asterisco, un nome prende **tutto il resto**, come lista:

```
primo, *resto = temperature          # 3 e [-1, -4, 0, 2, 5, -2]
*inizio, ultimo = temperature        # [3, -1, -4, 0, 2, 5] e -2
```

L'asterisco funziona anche al contrario, nella chiamata di una funzione: `zip(*tabella)` passa a `zip` le righe della tabella come argomenti separati, e il risultato sono le colonne.

```
tabella = [[1, 2, 3], [4, 5, 6]]
list(zip(*tabella))                  # [(1, 4), (2, 5), (3, 6)]
```

È la trasposizione della comprehension illeggibile di prima, in una riga che si legge: «metti insieme gli elementi nella stessa posizione di ogni riga».

Parte III — Dati veri

Una lista di dizionari

I dati che arrivano da un file, da un foglio di calcolo o dal web hanno quasi sempre questa forma: **una riga per elemento, un campo per nome**. In Python è una lista di dizionari, tutti con le stesse chiavi:

```
dati = [{'stazione': 'Brera',    'mese': 'gen', 'mm': 58},
        {'stazione': 'Brera',    'mese': 'feb', 'mm': 49},
        {'stazione': 'Lambrate', 'mese': 'gen', 'mm': 61},
        {'stazione': 'Lambrate', 'mese': 'feb', 'mm': 52}]
```

Rispetto a una lista di liste, ogni valore si raggiunge **per nome**: `riga['mm']` invece di `riga[2]`. Così si smette di ricordare in quale colonna stia che cosa e di saltare la colonna del nome: è il problema della tabella della Florida e di `primoMese`, risolto dalla forma dei dati.

Una tabella che arriva come lista di liste, con l'intestazione a parte, si trasforma con `zip`:

```
intestazione = ['stazione', 'mese', 'mm']
righe = [['Brera', 'gen', 58], ['Brera', 'feb', 49],
         ['Lambrate', 'gen', 61], ['Lambrate', 'feb', 52]]

dati = []
for riga in righe:
    dati.append(dict(zip(intestazione, riga)))
```

Poi si filtra e si aggrega con gli strumenti di sempre:

```
[r['mm'] for r in dati if r['mese'] == 'gen']        # [58, 61]: la pioggia di gennaio

totali = {}
for r in dati:
    totali[r['stazione']] = totali.get(r['stazione'], 0) + r['mm']
# {'Brera': 107, 'Lambrate': 113}
```

Questa è anche la forma del **JSON**, il formato con cui si scambiano dati sul web: in un file JSON una lista di dizionari si scrive quasi uguale a come si scrive in Python. Leggere un file arriva in `L09`; e in `L10` pandas tratterà questa stessa tabella come un `DataFrame`, dove «filtra le righe di gennaio» e «somma per stazione» diventano un'istruzione ciascuno.

Che cosa resta fuori

Alcuni strumenti vicini a questi richiedono di saper scrivere una **funzione**, e arrivano dopo `L08`: le funzioni senza nome (`lambda`), l'argomento `key=` per ordinare in base a un criterio scelto da noi, le funzioni che producono valori uno alla volta (`yield`), e le annotazioni di tipo.